CSE 4304-Data Structures Lab. Winter 2022
Date: Sept 06, 2022.
Target Group: All
Topic: Linked lists

Instructions:

- Task naming format: fullID_T01L05_2A.c/CPP
- If you find any issues in problem description/test cases, comment in the google classroom.
- If you find any tricky test case that I didn't include but others might forget to handle, please comment! I'll be happy to add.
- Use appropriate comments in your code. This will help you to recall the solution in the future easily.
- Obtained marks will vary bQased on the efficiency of the solution.
- Do not use <bits/stdc++.h> library, add the necessary libraries one after another.
- Do not use any built-in functions or any STL libraries. If the solution is extremely difficult without the usage of such components, discuss it with the instructors.
- Modified sections will be marked with BLUE color.

Group	Tasks
1B	1 2 3 4
2B	1 2 3 4
2A	1 2 5 6
1A	1 2 5 7

Task-01:

Implement the basic operations using a Linked list. Your program should include the following functions:

1. **Insert_front**(int key):
    - Insert the element with the 'key' at the beginning of the list.
    - Time Complexity: O(1)
2. **Insert_back**(int key):
    - Insert the element with the 'key' at the end of the list.
    - Time Complexity: O(1)
3. **Insert_after_node** (int key, int v):
    - Insert a node with the 'key' after the node containing the value 'v' if it exists. (shows error message otherwise).
    - Time complexity: $O(n)$
4. **Update_node** (int key, int v):
    - Looks for the node with value *v* and updates it with the new value 'key' (error message if the node doesn't exist)
    - Time complexity: O(n)
5. **Remove_head** ():
    - Remove the first node from the linked list.
    - Time complexity: O(1)
6. **Remove_element** (int key):
    - Removes the node containing the 'key' if it exists (else throw an error message).
    - Time complexity O(n)
7. **Remove_end** ():
    - Remove the last node from the linked list.
    - Time complexity: O(n)

Input format:

- The program will offer the user the following operations (as long as the user doesn’t use option 7):
    - Press 1 to insert at front
    - Press 2 to insert at back
    - Press 3 to insert after a node
    - Press 4 to update a node
    - Press 5 to remove the first node
    - Press 6 to remove a node
    - Press 7 to remove the last node
    - Press 8 to exit.
- After the user chooses an operation, the program takes necessary actions (or asks for further info if required).

Output format:

- After each operation, the status of the list is printed.

Task 2

- Satisfy the requirements of Task-1 using **'Doubly linked list'**.
- One additional requirement is, that after each operation, you have to print the linked list twice:
  - From head to tail.
  - From the tail towards the head (don't use recursive implementation, rather utilize the 'previous' pointers).
- The **Remove_end()** function should be done in **O(1)**

Task 3

Implement a ***'Deque'*** using ***'Double linked list'***. Your program should offer the user the following options:

1. void **push_front**(int key): Insert an element at the beginning of the list.
2. void **push_back**(int key): Insert an element at the end of the list.
3. int **pop_front**(): Extracts the first element from the list.
4. int **pop_back**(): Extracts the last element from the list.
5. int **size()**: Returns the total number of items in the Deque.

Note:

- The maximum time complexity for any operations is **O(1)**.
- For options 3,4: the program shows an error message if the list is empty.

Input format:

- The program will offer the user the following operations (as long as the user doesn't use option 6):
  - Press 1 to push_front
  - Press 2 to push_back
  - Press 3 to pop_front
  - Press 4 to pop_back
  - Press 5 for size
  - Press 5 to exit.
- After the user chooses an operation, the program takes necessary actions (or asks for further values if required).

Output format:

- After each operation, the status of the list is printed.

Input	**Output**
1 10	10
1 20	20 10
2 30	20 10 30
5	3
2 40	20 10 30 40
3	10 30 40
1 50	50 10 30 40
4	50 10 30
5	3

Task 4

A set of sorted numbers is stored in a linked list. Your task is to keep the first occurrence of a number and remove the other duplicate values from the list.
(Stop taking input when the user enters -1)

Input	**Output**
2 7 7 10 12 18 25 25 25 27 -1	2 7 10 12 18 25 27
5 5 5 5 5 -1	5
1 2 3 4 5 -1	1 2 3 4 5
10 20 20 20 20 20 20 -1	10 20

Note: Your solution should remove the node from the existing linked list instead of using a new linked list to store unique elements.

Task 5

Given two sorted linked lists in increasing order, your task is to create a **new linked list** that stores the item that intersects in both lists.

The first two lines represent the two lists (input stops with -1). Store them in a linked list, and show their intersection as output. Print 'empty' if there is no intersection.

Input	**Output**
1 2 3 4 5 6 -1 3 4 5 7 8 -1	3 4 5
10 20 30 40 50 60 70 80 90 -1 1 5 10 15 30 25 40 43 77 80 99 -1	10 30 40 80
2 2 3 3 3 5 5 5 5 5 9 9 9 -1 0 1 2 3 4 5 6 7 -1	2 3 5
1 2 3 4 5 -1 6 7 8 9 -1	Empty
6 7 8 9 -1 1 2 3 4 5 -1	Empty

Task 6 Implementing the basic operations of **Stack with Linked lists**

Stacks is a linear data structure that follows the Last In First Out (LIFO) principle. The last item to be inserted is the first one to be deleted. For example, you have a stack of trays on a table. The tray at the top of the stack is the first item to be moved if you require a tray from that stack.

The Insertion and Deletion of an element from the stack are a little bit different from the traditional operation. We define the two corresponding operations as Push() and Pop() from the stack.

The first line contains *N* representing the size of the stack. The lines contain the 'function IDs' and the required parameter (if applicable). Function ID 1, 2, 3, 4, 5, and 6 corresponds to push, pop, isEmpty, isFull ~~(assume max size of Stack=5)~~, size, and top. The return type of inEmpty and isFull is Boolean. Stop taking input once given -1.

Input	**Output**
5 3 2 1 10 1 20 5 1 30 6 2 1 40 1 50 4 1 60 4 5 1 70 5 2 6 -1	 True Underflow 10 10 20 2 10 20 30 30 10 20 10 20 40 10 20 40 50 False 10 20 40 50 60 False 5 10 20 40 50 60 70 6 10 20 40 50 60 60

Task 7

Given a list of random integers stored in a ‘linked list’, your task is to sort them using bubble sort.

Input	**Output**
5 3 1 9 8 2 4 7 -1	1 2 3 4 5 7 8 9
5 1 4 2 8 -1	1 2 4 5 8
5 1 4 2 8 8 2 4 1 5 -1	1 1 2 2 4 4 5 5 8 8